Portable Spec Kit — Spec-Persistent Development

Technical Overview — Architecture Reference Document

A comprehensive technical document covering the framework's architecture, user profile system, flow documentation, agent compatibility, security model, testing infrastructure, and deployment strategy.

v0.4.6 • April 2026

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Computer Science — Artificial Intelligence

Version: v0.4.6

Tests: 676 passing (531 framework + 145 benchmarking), 43 sections

Executive Summary

Portable Spec Kit is a lightweight, zero-install, personalized spec-persistent development framework for AI-assisted engineering. It consists of a single markdown file (`portable-spec-kit.md`) that any AI coding agent reads to follow consistent engineering standards, manage specifications, track tasks, and maintain context across sessions.

> **Key numbers:** 1 file, 5 AI agents supported, 10 core strengths, 8 source code templates, 13 flow documents, 676 automated tests (531 framework + 145 benchmarking), 43 test sections.

Key Highlights

Metric	Value
Current version	v0.4.6
Framework size	Single markdown file (~1,180 lines)
Dependencies	Zero
Install time	Seconds (one curl command)
AI agents supported	Claude, Copilot, Cursor, Windsurf, Cline
Core strengths	12 (Lightweight, Portable, Personalized, Spec-Persistent, Agent-Agnostic, Context-Persistent, Secure, Non-Blocking, Self-Scaffolding, Self-Validating, Automated Test Generation, CI/CD Ready)
Source code templates	8 (Web, Python, Mobile, Android, iOS, Full Stack, Full Stack + Mobile, Document)
Automated tests	676 (531 framework + 145 benchmarking)
Research	SPD concept paper (5,300 words), benchmarking report, agentic communication discovery
Flow documents	13 step-by-step work flows

Project scenarios	9 (new, existing, monorepo, partial, cloned, subdir, returning, multiple, kit in one dir)
Profile setup	One-time, GitHub auto-detect, 3 preference questions, per-user workspace files
Versioning	Two-level: framework v0.x.x (patches) + release v0.x (milestones)

Version Changelog

v0.4 — CI/CD Pipeline + Spec-Based Test Generation (April 2026)

Kit: v0.4.1 — v0.4.6

- **GitHub Actions CI:** `ci.yml` runs all 3 test suites on every push and PR. `release.yml` verifies tag matches framework version on v* tag push.
- **Framework CI/CD rules:** `ci.yml` template with stack-aware commands (Jest/pytest/Go/Bash), New Project Setup Step 7.5, Existing Project Setup checklist CI item, CI & Community Contributions section (4 rules).
- **Spec-based test generation (8 rules):** SPECS origin detection (forward vs retroactive), per-feature `### F{n}` acceptance criteria format, stub generation trigger, stack-aware stub formats, stub completion gate.
- **check_stub_complete():** added to `tests/test-release-check.sh` (kit copy + template) — blocks release if test stubs have unfilled TODO markers.
- **Community files:** PR template, bug report + feature request issue templates. CI badge in README.
- **Cross-platform sed fix:** `test-spd-benchmarking.sh` now runs on Ubuntu (GitHub Actions).
- **676 tests** (531 framework across 43 sections + 145 benchmarking). Section 42: CI/CD (29 tests). Section 43: Spec-Based Test Generation (15 tests).
- **v0.4.5:** 2 new flow docs (11-existing-project-setup, 12-cicd-setup). All 12 flow docs validated at 63-char box alignment. Section 19 expanded to cover all 12 flows. ARD flow diagrams section added. Prepare release rule updated: flow docs first, mandatory ARD audit, mandatory PDFs.
- **v0.4.6:** Flow docs reordered logically (onboarding 01-06, session 07-10, development 11-12, release 13). New `05-project-init.md` — explicit `init` / `reinit` commands. Renamed: `12-project-lifecycle.md`, `13-release-workflow.md`. `init` / `reinit` added as explicit command signals with full flows. Test Execution Flow extracted as named section — referenced by all test-running commands. Version bump BEFORE push rule added. 676 tests (531 framework + 145 benchmarking).

v0.3 — Framework Hardening + R→F→T Traceability (April 2026)

Kit: v0.3.1 — v0.3.27

- **R→F→T traceability chain:** every done feature requires test reference in SPECS.md before release. `tests/test-release-check.sh` distributed as kit template, auto-created on project setup.

- **15 new enforcement sections:** SPECS.md staleness check, RELEASES.md trigger rule, scope change recording (DROP/ADD/MODIFY/REPLACE), no-slip task rule, pipeline sync, pre-release consistency gate (Section 41).
- **Release Process rule — full 8-step sequence:** tests → counts → version bump → PDFs → RELEASES.md → CHANGELOG.md → GitHub release → tag update. Explicit signals only.
- **Release command aliases:** `update release` = alias for `prepare release` (full 8-step); `refresh release` = re-test + sync current release without bumping version.
- **Release notes scope rule:** only include changes committed and visible in public repo; never mention excluded files (private docs, research papers).
- **Simplified versioning:** aligned patches (v0.N.x = release v0.N). All specific version examples replaced with generic `v0.N.x` / `v0.1.4→v0.1.5` so rules never go stale. GitHub release title = minor version (v0.N — Title) matching CHANGELOG.
- **README hardening:** Critical Scenarios table (8 situations), Development Guidelines section, "first methodology native to the AI era" framing.
- **676 tests** (531 framework across 43 sections + 145 benchmarking). All passing.

v0.2 — Versioning + Project Scenarios (April 2026)

Kit: v0.2.1 — v0.2.9

- **Aligned versioning:** v0.N.x patches belong to release v0.N (e.g. v0.2.x = release v0.2). Auto-restructure agent files when framework version changes on pull.
- **Existing project setup:** guide don't force — scan, show checklist, user picks changes. Never rename/move/delete without approval.
- **9 project scenarios:** new, existing, monorepo, partial, cloned, subdir, multiple projects, returning, kit in one subdir only.
- **Project directory confirmation:** list directories, user picks or types new path before creating agent/ files.
- **Git rule:** check .git/ before committing — each project can be its own repo.
- **Version-based TASKS.md:** tasks grouped under release headings (v0.0 Done, v0.1 Done, v0.2 Done, v0.3 Current, Backlog).
- **RELEASES.md:** kit version range per release (v0.0.x for v0.0, v0.1.x for v0.1).
- **AGENT_CONTEXT.md:** tracks Kit version — compared on session start to trigger restructure.
- **442 framework tests across 41 sections + 145 benchmarking tests (587 total). Added: Python/Conda env rules (27 tests), README validation (3 tests), requirements-to-delivery flow checks, 15 enforcement sections (staleness, RELEASES trigger, scope changes, no-slip, pipeline sync, security, push rules, git, existing projects, scenarios, retro fill, context persistence across 10 disruptions, R→F→T traceability chain).**

v0.1 — Personalized Profile + Flow Documentation (April 2026)

Kit: v0.1.1 — v0.1.9

User Profile

- **Profile moved to** `.portable-spec-kit/user-profile/user-profile-{username}.md`
- **Global home directory storage — asked once, works everywhere**
- **Per-user workspace files — committed, team-ready**
- **Username detection via** `git config user.name` **(slugified)**
- **First-time setup: GitHub fetch + 3 questions with RECOMMENDED defaults + Enter to skip**
- **New project: show profile, keep or customize, save to workspace**
- **Returning session: workspace exists → silent; only global → show/confirm**
- **12 edge cases handled (no gh CLI, empty files, multiple users, special chars)**
- **Cross-OS: macOS/Linux (** `~` **) + Windows (** `%USERPROFILE%` **)**

Flow Documentation

- **13 flow documents created in** `docs/work-flows/`
- **Each flow: title, trigger condition, step-by-step diagram, edge cases, files involved**
- **Context Management rule: update flows after implementation/tests**

Framework

- **10 core strengths (added Personalized, Secure, restored Self-Scaffolding)**
- **8 source code structure templates (added Mobile, Android Native, iOS Native, Full Stack + Mobile)**
- **Strict security rule — never expose API keys even if explicitly asked**
- **Context triad: context + flows + tests must always match code**
- **Guide moved from** `docs/` **to** `ard/`
- **Deleted** `setup.sh` **— agent handles profile creation**

Agent-Agnostic

- **Removed all** `.claude/` **paths,** `noreply@anthropic.com` **,** `Claude Opus`
- **Generic** `Co-Authored-By: AI Agent <noreply@ai-agent.dev>`
- **27 agent-switching tests verifying zero data loss**

Testing

- 242 tests across 22 sections (was 122)
- New sections: User Profile directory, first-time setup, new project, returning session, edge cases, flow documentation, ARD directory, context management

v0.0 — Initial Release (March 2026)

- CLAUDE.md framework — portable spec-persistent development
- README with spec-kit comparison table
- CONTRIBUTING.md with PR guidelines
- PDF Quick Guide
- Two example projects (starter + my-app)
- GitHub repo created (aqibmumtaz/portable-spec-kit)
- Sync script for publishing
- 122 automated tests

Table of Contents

1. Architecture Overview
2. User Profile System
3. Work Flows
4. Multi-Agent Support
5. Security Model
6. Source Code Templates
7. Spec-Persistent Development Pipeline
8. Testing Infrastructure
9. File Structure & Purposes
10. Deployment & Syncing

1. Architecture Overview

The framework is a single markdown file that AI agents read as their instruction set. It creates a layered system:

```
Layer 1: portable-spec-kit.md          ← Framework (portable, shared)
Layer 2: .portable-spec-kit/           ← User profile (per-user, committed)
Layer 3: agent/AGENT.md                ← Project rules (per-project)
Layer 4: agent/AGENT_CONTEXT.md        ← Living state (per-session)
Layer 5: agent/SPECS → PLANNING → TASKS → TRACKER  ← Dev pipeline
```

Design Principles

Principle	Implementation
Zero install	One curl command downloads the file, symlinks for all agents
Personalized	GitHub profile auto-detect, 3 preference questions, adapts behavior
Agent-agnostic	Symlinks: CLAUDE.md, .cursorrules, .windsurfrules, .clinerules, copilot-instructions.md
Non-blocking	Code first, specs retroactively — agent tracks silently
Context-persistent	AGENT_CONTEXT.md updated every session, picks up after weeks
Self-validating	Agent tests everything, fixes before presenting, 98%+ coverage target
Secure	Never expose API keys — even if asked. Absolute rule, no exceptions.

File Hierarchy

```
workspace/
├── portable-spec-kit.md                ← Framework (source)
├── .portable-spec-kit/                 ← Kit config
│   └── user-profile/
│       └── user-profile-{username}.md  ← Per-user profile (committed)
```

```
├── CLAUDE.md → portable-spec-kit.md      ← Claude Code symlink
├── .cursorrules → portable-spec-kit.md   ← Cursor symlink
├── .windsurfrules → portable-spec-kit.md ← Windsurf symlink
├── .clinerules → portable-spec-kit.md    ← Cline symlink
├── .github/copilot-instructions.md → ../ ← Copilot symlink
├── WORKSPACE_CONTEXT.md                  ← Workspace state
├── agent/                                ← Project management
│   ├── AGENT.md                          ← Project rules, stack
│   ├── AGENT_CONTEXT.md                  ← Living state
│   ├── SPECS.md                          ← Requirements
│   ├── PLANS.md                       ← Architecture
│   ├── TASKS.md                          ← Task tracking
│   └── RELEASES.md                        ← Version history
├── src/                                  ← Source code
├── tests/
│   └── test-release-check.sh             ← R→F→T validator (auto-created by kit)
├── docs/work-flows/                           ← System flow documentation
└── ard/                                  ← Architecture Reference Documents
```

2. User Profile System

Purpose

Tells the AI agent WHO it's working with — affecting response depth, technical language, analogies, and autonomy level.

Storage

Location	Purpose	Committed?
`~/.portable-spec-kit/user-profile/user-profile-{username}.md`	**Global — asked once**	**No (home dir)**
`workspace/.portable-spec-kit/user-profile/user-profile-{username}.md`	**Per-project, per-user**	**Yes**

Lookup Order

1. **Workspace** `.portable-spec-kit/user-profile/user-profile-{username}.md`
2. **Global** `~/.portable-spec-kit/user-profile/user-profile-{username}.md`
3. **Neither → first-time setup**

Username Detection

`git config user.name` **→ slugified (lowercase, spaces → dashes). Used for filename.** `gh api user` **used only for greeting name/bio.**

Profile Format

```
# User Profile
> Auto-created on first session. Edit anytime.

- **Name** — Education. Expertise.
- Communication style: {selected or custom}
- Working pattern: {selected or custom}
- AI delegation: {selected or custom}
```

Preference Options

Question	Options
Communication	(a) direct and concise ← RECOMMENDED (b) direct, data-driven, comprehensive (c) conversational and collaborative
Working pattern	(a) iterative — starts brief, expands ← RECOMMENDED (b) plan-first — full specs before code (c) prototype-fast — build quick, refine
AI delegation	(a) AI does 70%, user guides 30% ← RECOMMENDED (b) AI does 90%, user reviews 10% (c) 50/50 collaboration

Edge Cases

Scenario	Handling
No gh CLI	Ask name/expertise manually
GitHub name empty	Use login as fallback
GitHub bio empty	Ask for education/expertise
Empty profile file	Treat as missing, run setup
Multiple users (team)	Each user has own file
Special chars in name	Slugified for filename
Agent can't write	Show content, user creates manually
User skips all questions	Recommended defaults applied

3. Work Flows

13 documented flows in `docs/work-flows/` covering every system interaction:

#	Flow	Trigger	Key Actions
01	First Session in New Workspace	New workspace	Environment detection → WORKSPACE_CONTEXT → auto-scan
02	User Profile Setup	First time using kit	GitHub fetch → 3 questions → save global + workspace
03	New Project Setup	Creating new project	Show profile → keep/customize → scaffold agent/ + dirs
04	Existing Project Setup	First-time kit onboarding	Detect state → scan → fill agent/ → checklist → apply selected
05	Project Init & Reinit	`init` / `reinit` commands	Confirm dir → deep scan → create/update agent/ files → show summary
06	CI/CD Setup	New/existing project setup	Stack detect → ci.yml template → badge → branch protection
07	Returning Session	Coming back to project	Load profile → read context → summarize → continue
08	Agent Switching	Changing AI agent	All agents read same file via symlinks → zero data loss
09	Profile Customization	Different prefs per project	Show CURRENT + RECOMMENDED → save local override
10	File Management	Agent encounters managed file	Create / restructure / leave-as-is — never lose content
11	Spec-Persistent Development	Any workflow — specs persist	SPECS → PLANS → TASKS → RELEASES — living specs, never block
12	Project Lifecycle		

		Client requirements received	R→F traceability → scope changes → F→T coverage → handoff
13	Release Workflow	prepare release command	Tests → counts → version bump → PDFs → publish → tag

Context Management Rule

> **After completing implementations or running tests: update `agent/AGENT_CONTEXT.md` to reflect current code status. Also update `docs/work-flows/` if any work flow changed, and update test files if new behaviors were added. Context, flows, and tests must always match the actual state of the code.**

Flow Diagrams

Step-by-step box diagrams for all 13 flows. All box lines are exactly 63 display characters wide. Connectors: `└──────┬──────┘` → `┌──────▼──────┐`. No tree-style standalone arrows or spacing lines.

01 — First Session in New Workspace

When: User opens an AI agent in a workspace for the first time — no WORKSPACE_CONTEXT.md exists.

```
┌──────────────────────────────────────────────────────────────┐
│  1. CHECK PROFILE                                            │
│     workspace/.portable-spec-kit/user-profile/ → FOUND?      │
│     ~/.portable-spec-kit/user-profile/ → FOUND?              │
│     Neither → run User Profile Setup flow                    │
│                                                              │
│     NOTE: Profile setup and workspace scan are independent   │
│     If user skips profile → apply defaults, proceed          │
│     Never block workspace scan waiting for profile           │
└──────────────────────┬───────────────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────────────┐
│  2. DETECT ENVIRONMENT                                       │
│     OS, Node version, Python version, tools installed        │
│     → populate Environment & Tools section                   │
└──────────────────────┬───────────────────────────────────────┘
                       │
```

```
┌──────────────────────▼──────────────────────────────────────┐
│  3. SCAN WORKSPACE                                          │
│     List all directories → identify projects                │
│     → populate Workspace Overview table                     │
│     Announce: "Scanning workspace — found X projects..."    │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  4. CREATE WORKSPACE_CONTEXT.md                             │
│     Sections:                                               │
│     - Workspace Overview (table of all projects)            │
│     - Environment & Tools                                   │
│     - Key Conventions                                       │
│     - Last Updated                                          │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  5. CREATE AGENT/ DIRS FOR PROJECTS WITHOUT THEM            │
│     For each project found without agent/ dir:              │
│     - Create agent/ with 6 template files                   │
│     - Apply File Creation/Update Rule to README.md          │
│     Show kit status: 🔍 Understanding your project...       │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  6. WORKSPACE READY                                         │
│     "Workspace scanned. X projects found."                  │
│     "What would you like to work on?"                       │
└──────────────────────────────────────────────────────────────┘
```

02 — User Profile Setup

When: First time using the kit on any machine — no profile exists anywhere.

```
┌──────────────────────────────────────────────────────────────┐
│  1. DETECT USERNAME                                          │
│     git config user.name → slugified (lowercase, spaces→-)   │
│     Filename: user-profile-jane-smith.md                     │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  2. FETCH GITHUB PROFILE                                     │
│     gh api user → full name, bio (if gh authenticated)       │
│     ├─ gh available → "Welcome, Jane Smith!"                 │
│     ├─ gh not authenticated → ask name + expertise manually  │
│     └─ gh not installed → ask manually                       │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  3. ASK 3 PREFERENCE QUESTIONS                               │
```

```
│     Q1: Communication style?                                    │
│         (a) direct and concise ← RECOMMENDED                    │
│         (b) data-driven, comprehensive with tables              │
│         (c) conversational and collaborative                    │
│         Enter = use recommended                                 │
│                                                                 │
│     Q2: Working pattern?                                        │
│         (a) iterative ← RECOMMENDED                             │
│         (b) plan-first                                          │
│         (c) prototype-fast                                      │
│         Enter = use recommended                                 │
│                                                                 │
│     Q3: AI delegation?                                          │
│         (a) AI does 70%, user guides 30% ← RECOMMENDED          │
│         (b) AI does 90%, user reviews 10%                       │
│         (c) 50/50 collaboration                                 │
│         Enter = use recommended                                 │
└─────────────────────────┬───────────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────────┐
│  4. CONFIRM PROFILE                                             │
│     ┌──────────────────────────────────┐                        │
│     │ Jane Smith — B.S. CS.            │                        │
│     │ Communication: direct            │                        │
│     │ Working pattern: iterative       │                        │
│     │ AI delegation: AI does 70%       │                        │
│     └──────────────────────────────────┘                        │
│     "Looks good? (Enter = yes, or type changes)"                │
└─────────────────────────┬───────────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────────┐
│  5. SAVE PROFILE                                                │
│     ~/.portable-spec-kit/user-profile/user-profile-{u}.md       │
│     workspace/.portable-spec-kit/user-profile/...               │
│     → won't ask again on this machine                           │
└─────────────────────────┬───────────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────────┐
│  6. CONTINUE TO PROJECT SETUP                                   │
│     ✓ Profile ready — proceeds to workspace/project scan        │
└─────────────────────────────────────────────────────────────────┘
```

03 — New Project Setup

When: User asks to create a new project or enters a directory without agent/files.

```
┌─────────────────────────────────────────────────────────────────┐
│  1. LOAD USER PROFILE                                           │
│     workspace/.portable-spec-kit/ → global → setup flow         │
```

```
│     Show: "Welcome back, Jane! Setting up new project."     │
│     Keep or customize for this project?                     │
└─────────────────────────┬───────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────┐
│  2. CREATE PROJECT STRUCTURE (immediately — no questions)   │
│     agent/ + 6 files, src/, tests/, docs/, ard/             │
│     input/, output/, cache/                                 │
│     README.md, .gitignore, .env.example                     │
│     tests/test-release-check.sh (R→F→T validator)           │
└─────────────────────────┬───────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────┐
│  3. REPORT TO USER                                          │
│     Show structure created — wait for user to start         │
└─────────────────────────┬───────────────────────────────────┘
                          │  When user is ready:
┌─────────────────────────▼───────────────────────────────────┐
│  4. SPECS DISCUSSION                                        │
│     Write agent/SPECS.md — requirements, features           │
│     Write per-feature acceptance criteria (### F{n})        │
│     Agent generates test stubs for forward-flow features    │
└─────────────────────────┬───────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────┐
│  5. TECH STACK → user approves                              │
│     Write agent/PLANS.md — architecture, phases             │
└─────────────────────────┬───────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────┐
│  6. INITIALIZE STACK                                        │
│     Install deps, update .gitignore, assign port            │
│     If Python → conda environment selection flow            │
└─────────────────────────┬───────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────┐
│  7. CREATE CI WORKFLOW                                      │
│     .github/workflows/ci.yml (stack-aware test command)     │
│     Always includes test-release-check.sh as final step     │
│     Add CI badge to README.md                               │
│     "CI will run on every push and PR."                     │
└─────────────────────────┬───────────────────────────────────┘
                          │
┌─────────────────────────▼───────────────────────────────────┐
│  8. START DEVELOPMENT                                       │
│     Update agent/TASKS.md → begin building                  │
└─────────────────────────────────────────────────────────────┘
```

05 — Project Init & Reinit

When: User says "init" (full project scan, any kit status) or "reinit" (re-scan to sync stale agent files). Agent never auto-triggers either command.

```
┌──────────────────────────────────────────────────────────┐
│  "init" — Full Scan (New / Partial / Mapped)              │
│     Confirm project directory                             │
│     Deep scan — config files, src/, top-level dirs        │
│     Create / update all 6 agent/ files from scan          │
│     Present optional changes checklist                    │
│     Apply selected → show init summary ✅                 │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│  "reinit" — Re-scan (already Mapped)                      │
│     Read agent/AGENT.md + AGENT_CONTEXT.md as baseline    │
│     Deep scan — source files, config, directory structure │
│     Update AGENT.md — only fields that changed            │
│     Rebuild AGENT_CONTEXT.md from current codebase        │
│     SPECS.md staleness check (completed tasks vs features) │
│     PLANS.md vs code check — flag stale architecture      │
│     Show reinit summary ✅                                │
└──────────────────────────────────────────────────────────┘
```

10 — File Management

When: Agent encounters any auto-managed file (WORKSPACE_CONTEXT.md, README.md, agent/ files) and needs to decide whether to create, update, or leave it.

```
┌──────────────────────────────────────────────────────────┐
│  ENCOUNTER A MANAGED FILE                                 │
│  (WORKSPACE_CONTEXT.md / README.md / agent/* files)       │
└──────────────────────┬───────────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────────┐
│  Does file exist?                                         │
│     ├─ NO  → CREATE from template                         │
│     │        Fill in known details (project name, stack)  │
│     └─ YES → Does file match template?                    │
│              ├─ YES → LEAVE AS-IS — no changes            │
│              └─ NO  → RESTRUCTURE:                        │
│                       Read ALL content                    │
│                       Map to template sections            │
│                       Reorganize structure                │
│                       RETAIN every detail                 │
└──────────────────────────────────────────────────────────┘
```

Kit Version Update — Full Scan Flow (runs when kit version changes on pull):

```
┌──────────────────────────────────────────────────────────────┐
│  1. RESTRUCTURE AGENT/ FILES                                 │
│     Compare each file to current template                    │
│     Reorganize structure, retain all existing content        │
│     Update **Kit:** field to new version                     │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  2. STALE FIELD NAME SWEEP                                   │
│     grep -r "Framework versions:" → rename to "Kit:"         │
│     grep -r "**Framework:**" → rename to "**Kit:**"          │
│     Check ALL files: agent/, docs/, examples/, templates     │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  3. SCAN PROJECT CODEBASE                                    │
│     Read source files, config files, directory structure     │
│     (package.json, requirements.txt, Dockerfile, etc.)       │
│     Update AGENT.md: stack, tech, ports — from actual code   │
│     Update AGENT_CONTEXT.md: phase, done, next — from state  │
└──────────────────────┬───────────────────────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────────────┐
│  4. SHOW COMBINED SUMMARY                                    │
│     "Portable Spec Kit updated to vX.X."                     │
│     "Your project: [stack] · [phase] · [X tasks pending]"    │
│     "Agent files updated: AGENT.md, AGENT_CONTEXT.md"        │
│     "What's new in vX.X: [CHANGELOG entries]"                │
│     Continue conversation — zero interruption                │
└──────────────────────────────────────────────────────────────┘
```

09 — Profile Customization

When: User wants different preferences for a specific project.

```
┌──────────────────────────────────────────────────────────────┐
│  1. SHOW CURRENT PROFILE                                     │
│     Load global: ~/.portable-spec-kit/user-profile/...       │
│     ┌──────────────────────────────┐                         │
│     │ Jane Smith — B.S. CS.        │                         │
│     │ Communication: direct        │                         │
│     │ Working pattern: iterative   │                         │
│     │ AI delegation: AI does 70%   │                         │
│     └──────────────────────────────┘                         │
│     "Keep or customize? (Enter = keep)"                      │
│     (a) Keep as-is  (b) Customize for this project           │
└──────────────────────┬───────────────────────────────────────┘
```

```
                        │ User selects (b)
┌──────────────────────▼──────────────────────────────────────┐
│  2. ASK 3 QUESTIONS — each shows CURRENT and RECOMMENDED    │
│     Q1: Communication style?                                 │
│     Q2: Working pattern?                                     │
│     Q3: AI delegation?                                       │
│     Press Enter to keep current value for each               │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  3. CONFIRM CHANGES                                          │
│     Show summary with changed vs unchanged fields            │
│     "Looks good? (Enter = yes, or type changes)"             │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  4. SAVE TO WORKSPACE — global unchanged                     │
│     workspace/.portable-spec-kit/user-profile/...            │
│     "This workspace uses local profile from now on"          │
└──────────────────────────────────────────────────────────────┘
```

07 — Returning Session

When: User opens an AI agent in an existing project — coming back after hours, days, weeks, or months.

```
┌──────────────────────────────────────────────────────────────┐
│  1. LOAD FRAMEWORK                                           │
│     Agent reads portable-spec-kit.md (via symlink)           │
│     Check kit version: Framework Version in kit              │
│     vs **Kit:** in agent/AGENT_CONTEXT.md                    │
│     ├─ Same → proceed normally                               │
│     └─ Different → run Kit Update flow (see flow 04)         │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  2. SHOW KIT STATUS (once at session start)                  │
│     ✅ Spec Kit: Project mapped (vX.X.X) — reading context  │
│     ⚠️  Spec Kit: Partial context — filling in gaps...       │
│     🔍 Spec Kit: Understanding your project — scanning...   │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  3. LOAD PROFILE                                             │
│     1. workspace/.portable-spec-kit/user-profile/ → FOUND?  │
│        └─ Yes → load silently, no questions                  │
│     2. ~/.portable-spec-kit/user-profile/ → FOUND?           │
│        └─ Yes → show profile, keep or customize → save WS   │
│     3. Neither → run User Profile Setup flow first           │
└──────────────────────┬──────────────────────────────────────┘
```

```
                        │
┌──────────────────────▼──────────────────────────────────┐
│  4. READ PROJECT CONTEXT (in this order)                 │
│     1. agent/AGENT.md     — project rules, stack         │
│     2. agent/AGENT_CONTEXT.md — living state             │
│     3. agent/TASKS.md     — current tasks                │
│     4. agent/PLANS.md     — architecture                 │
└──────────────────────┬──────────────────────────────────┘
                        │
┌──────────────────────▼──────────────────────────────────┐
│  5. GREET + SUMMARIZE                                    │
│     "Welcome back, Jane! Here's where we left off:"      │
│     - Version and phase from AGENT_CONTEXT.md            │
│     - Tasks completed vs pending from TASKS.md           │
│     - Last decision + next step                          │
│     "Want to continue with [next task]?"                 │
└──────────────────────┬──────────────────────────────────┘
                        │
┌──────────────────────▼──────────────────────────────────┐
│  6. WORK                                                 │
│     User gives instructions → agent works                │
│     Track all tasks in TASKS.md (no-slip rule)           │
└──────────────────────┬──────────────────────────────────┘
                        │
┌──────────────────────▼──────────────────────────────────┐
│  7. SESSION END UPDATE                                   │
│     Update agent/AGENT_CONTEXT.md — progress, decisions  │
│     Update docs/work-flows/ if any flow changed          │
│     Verify no tasks slipped (scan conversation)          │
└──────────────────────────────────────────────────────────┘
```

11 — Spec-Persistent Development

When: User follows any development workflow — structured, agile, or mixed. The framework adapts to the user's style, not the other way around.

```
┌──────────────────────────────────────────────────────────┐
│  1. SPECS — Define what to build                         │
│     Write agent/SPECS.md — requirements (Rn), features (Fn) │
│     Per-feature acceptance criteria (### F{n} subsections)  │
│     Agent generates test stubs for forward-flow features  │
└──────────────────────┬──────────────────────────────────┘
                        │
┌──────────────────────▼──────────────────────────────────┐
│  2. PLANS — Architect how to build it                    │
│     Write agent/PLANS.md — stack, architecture, phases   │
│     Data model, API endpoints, security, methodology     │
└──────────────────────┬──────────────────────────────────┘
                        │
┌──────────────────────▼──────────────────────────────────┐
```

```
│  3. TASKS + BUILD — Execute                                    │
│     Write agent/TASKS.md (version-based, checkboxes)           │
│     For each task: TASKS.md FIRST → build → test → done        │
│     Stubs RED → implement → stubs GREEN → mark [x]             │
│     check_stub_complete() gate before marking [x]              │
└────────────────────────┬──────────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────────┐
│  4. SYNC — Keep all 4 pipeline files current                   │
│     SPECS.md, PLANS.md, TASKS.md, AGENT_CONTEXT.md             │
│     Record scope changes (DROP/ADD/MODIFY/REPLACE)             │
│     R→F traceability: Rn → Fn through every change             │
│     F→T traceability: Fn → test ref in Tests column            │
└────────────────────────┬──────────────────────────────────────┘
                         │ When all version tasks [x] done
┌────────────────────────▼──────────────────────────────────────┐
│  5. PREPARE RELEASE (10-step gate)                             │
│     Flow docs → counts → ARD audit → version bump              │
│     PDFs → RELEASES.md → CHANGELOG.md → publish                │
│     Section 41 (28 tests) catches any sync gaps                │
└────────────────────────┬──────────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────────┐
│  6. RELEASES.md — Record what happened                         │
│     Version summary, Kit range, changes by category            │
│     Test results, deployment, known issues                     │
└────────────────────────────────────────────────────────────────┘
```

12 — Project Lifecycle

When: A project goes through its full lifecycle — client requirements through delivery. Shows R→F→T traceability at every phase.

```
┌────────────────────────────────────────────────────────────────┐
│  1. CLIENT REQUIREMENTS                                        │
│     Client gives requirements (human language)                 │
│     Agent translates: Rn → Fn (R→F traceability starts)        │
│     SPECS.md + PLANS.md + TASKS.md + AGENT_CONTEXT.md          │
└────────────────────────┬──────────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────────┐
│  2. BUILD — features + scope changes                           │
│     Implement features — track in TASKS.md                     │
│     Client changes requirements mid-project:                   │
│     DROP / ADD / MODIFY / REPLACE — recorded in SPECS.md       │
│     R→F traceability maintained through every change           │
└────────────────────────┬──────────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────────┐
│  3. CONTEXT BREAK (developer stops / agent switch)             │
```

```
│     Context persists in agent/ files — nothing lost          │
│     Any agent / any session picks up from AGENT_CONTEXT.md   │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  4. RESUME                                                   │
│     Agent reads AGENT_CONTEXT.md → "Here's where we left"    │
│     Continues seamlessly — zero rebuild of context           │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  5. RELEASE v0.1                                             │
│     All version tasks [x] done → prepare release             │
│     RELEASES.md — what was delivered vs descoped             │
│     R→F→T traceability: every feature has test coverage      │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  6. HANDOFF (months later — new developer)                   │
│     Sarah reads agent/ files — understands everything        │
│     Requirements → features → decisions → test coverage      │
│     Continues without any briefing from original developer   │
└──────────────────────────────────────────────────────────────┘
```

13 — Release Workflow

When: User says "prepare release", "update release", or "refresh release". Runs the 10-step gate.

```
┌──────────────────────────────────────────────────────────────┐
│  1. DEVELOP                                                  │
│     Write code / add rules / update templates                │
│     Add tasks to TASKS.md as you go (no-slip rule)           │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  2. "run tests"                                              │
│     bash tests/test-spec-kit.sh          → 531 tests         │
│     bash tests/test-spd-benchmarking.sh  → 145 tests         │
│     bash tests/test-release-check.sh     → 57/57 features    │
│     Fix any failures before continuing                       │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  3. "prepare release" (10-step gate)                         │
│                                                              │
│  Step 1   Run all 3 suites — stop if any fail                │
│  Step 2   Update flow docs (FIRST) — any changed flows       │
│  Step 3   Counts + ARD audit — README badge, ard/ HTML       │
│           (version, flow count, test count, changelog)       │
```

```
│  Step 4   Bump version — AGENT_CONTEXT.md + README badge    │
│  Step 5   PDFs — open ard/*.html in browser → print to PDF  │
│  Step 6   Update agent/RELEASES.md                          │
│  Step 7   Update CHANGELOG.md                               │
│  Step 8   Publish GitHub Release (auto via gh if authed)    │
│  Step 9   After push — update v0.N tag to HEAD              │
│  Step 10  Show release summary block                        │
└─────────────────────┬──────────────────────────────────────┘
                      │
┌─────────────────────▼──────────────────────────────────────┐
│  4. "commit"                                                │
│     Stage and commit all changes                            │
└─────────────────────┬──────────────────────────────────────┘
                      │
┌─────────────────────▼──────────────────────────────────────┐
│  5. "push"                                                  │
│     bash agent/sync.sh "message"                            │
│     → copies to temp clone                                  │
│     → excludes docs/research/ (private)                     │
│     → pushes to aqibmumtaz/portable-spec-kit                │
│     → creates/updates GitHub Release from CHANGELOG.md      │
│     → updates v0.N tag to HEAD                              │
└─────────────────────┬──────────────────────────────────────┘
                      │
┌─────────────────────▼──────────────────────────────────────┐
│  6. VERIFY (after push)                                     │
│     CI badge turns green on GitHub (~2 min)                 │
│     GitHub Release shows correct notes                      │
│     v0.4 tag points to HEAD                                 │
└────────────────────────────────────────────────────────────┘
```

08 — Agent Switching

When: User switches from one AI agent to another (e.g., Claude → Cursor → Copilot).

```
┌────────────────────────────────────────────────────────────┐
│  DAY 1: AGENT A (e.g., Claude)                              │
│     Reads CLAUDE.md → symlink → portable-spec-kit.md        │
│     Reads user profile from .portable-spec-kit/user-profile/│
│     Reads agent/AGENT_CONTEXT.md                            │
│     Works on project → updates AGENT_CONTEXT.md, TASKS.md   │
└─────────────────────┬──────────────────────────────────────┘
                      │ User switches agent
┌─────────────────────▼──────────────────────────────────────┐
│  DAY 2: AGENT B (e.g., Cursor)                              │
│     Reads .cursorrules → same symlink → portable-spec-kit.md│
│     Reads SAME user profile                                 │
│     Reads SAME agent/AGENT_CONTEXT.md                       │
│     "Welcome back, Jane! Here's where we left off..."       │
```

```
│     Picks up exactly where Agent A stopped                    │
│     Works → updates SAME AGENT_CONTEXT.md, TASKS.md           │
└──────────────────────┬──────────────────────────────────────┘
                       │ User switches again
┌──────────────────────▼──────────────────────────────────────┐
│  DAY N: AGENT C (e.g., Copilot)                               │
│     Reads .github/copilot-instructions.md → same source       │
│     Reads SAME profile, SAME context                          │
│     Continues seamlessly — zero data loss                     │
└──────────────────────────────────────────────────────────────┘
```

04 — Existing Project Setup

When: User installs the spec kit on a project that already has code — first-time onboarding, not returning.

```
┌──────────────────────────────────────────────────────────────┐
│  0. DETECT PROJECT STATE (show once at session start)         │
│     Inspect agent/ directory:                                 │
│     ├─ Mapped   → ✅ Project mapped — read context normally   │
│     ├─ Partial  → ⚠  Partial context — fill gaps, skip scan   │
│     └─ New/None → 🔍 No context — proceed with full scan ↓    │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  1. ANNOUNCE SCAN                                             │
│     "Spec Kit is understanding your project —                 │
│      scanning structure, stack, files, dependencies..."       │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  2. SCAN THOROUGHLY                                           │
│     Config files:                                             │
│       package.json · requirements.txt · pyproject.toml        │
│       Dockerfile · docker-compose.yml · go.mod · Cargo.toml   │
│       tsconfig.json · pubspec.yaml · .env.example             │
│     Top-level dirs + sample src/ structure                    │
│     Build complete picture before touching anything           │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  3. FILL agent/AGENT.md FROM SCAN                             │
│     Stack, tech, dev server port, key scripts                 │
│     Env var names (.env.example — names only, never values)   │
│     If Python project → run Environment Selection flow        │
│     Never leave fields TBD if answer is visible in code       │
└──────────────────────┬──────────────────────────────────────┘
                       │
┌──────────────────────▼──────────────────────────────────────┐
│  4. FILL agent/AGENT_CONTEXT.md FROM SCAN                     │
```

```
│     Current phase, what appears done, key decisions          │
│     Directory structure and tech choices visible in code     │
│     Phase estimate from codebase state                       │
└──────────────────────────────────────────────────────────────┘
                        │
                        │
┌──────────────────────▼───────────────────────────────────────┐
│  5. PRESENT CHECKLIST                                        │
│     "Scan complete. Here's what I found and suggest:"        │
│                                                              │
│     Detected: [stack] · [version] · Port [N]                 │
│                                                              │
│     [x] Create agent/ (6 files, pre-filled from scan)        │
│     [x] Create WORKSPACE_CONTEXT.md                          │
│     [ ] Create .env.example from .env                        │
│     [ ] Create .github/workflows/ci.yml                      │
│     [ ] Restructure README.md to match template              │
│                                                              │
│     "Which changes? All, some, or none."                     │
└──────────────────────────────────────────────────────────────┘
                        │
                        │
┌──────────────────────▼───────────────────────────────────────┐
│  6. APPLY SELECTED CHANGES ONLY                              │
│     agent/ files — always safe to create                     │
│     Everything else — only if user selected it               │
│     Never rename / move / delete without explicit approval   │
└──────────────────────────────────────────────────────────────┘
                        │
                        │
┌──────────────────────▼───────────────────────────────────────┐
│  7. CONTINUE NORMALLY                                        │
│     Project is now spec kit managed                          │
│     Proceed to user's task — no more setup prompts           │
└──────────────────────────────────────────────────────────────┘
```

06 — CI/CD Setup

When: Agent creates a CI/CD pipeline — during new project setup (Step 7.5), existing project onboarding, or when user says "add CI".

```
┌──────────────────────────────────────────────────────────────┐
│  1. DETECT STACK                                             │
│     Read agent/AGENT.md — Stack table                        │
│     Identify: language, test runner, package manager         │
└──────────────────────────────────────────────────────────────┘
                        │
                        │
┌──────────────────────▼───────────────────────────────────────┐
│  2. SELECT TEST COMMAND (stack-aware)                        │
│     Node.js + Jest      → npx jest --passWithNoTests         │
│     Node.js + Vitest    → npx vitest run                     │
│     Node.js + generic   → npm test                           │
│     Python + pytest     → python -m pytest                   │
```

```
│     Go                  → go test ./...                    │
│     Bash scripts only   → (omit Run tests step)            │
│     Unknown             → echo 'Configure test command'    │
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│  3. GENERATE .github/workflows/ci.yml                      │
│     on: push (main) + pull_request                         │
│     runs-on: ubuntu-latest                                 │
│     steps:                                                 │
│       - actions/checkout@v4                                │
│       - {SETUP_STEPS} (Node/Python if applicable)          │
│       - run: {TEST_COMMAND}                                │
│       - run: bash tests/test-release-check.sh agent/SPECS.md│
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│  4. ADD CI BADGE TO README.md                              │
│     Insert as FIRST badge:                                 │
│     [![CI]({repo}/actions/workflows/ci.yml/badge.svg)]     │
│            ({repo}/actions/workflows/ci.yml)               │
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│  5. TELL USER: ENABLE BRANCH PROTECTION                    │
│     GitHub Settings → Branches → Add rule                  │
│     Require CI status checks to pass before merging        │
│     Prevents pushes with failing tests reaching main       │
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│  6. VERIFY (after first push)                              │
│     CI badge turns green on GitHub (~2 min)                │
│     Stack tests pass                                       │
│     R→F→T validator passes (0 done features = OK)          │
└────────────────────────────────────────────────────────────┘
```

4. Multi-Agent Support

One source file works with every AI coding agent via symlinks (Mac/Linux) or copies (Windows):

Agent	File Created	Method
Claude Code	`CLAUDE.md`	**Symlink → portable-spec-kit.md**
GitHub Copilot	`.github/copilot-instructions.md`	**Symlink → portable-spec-kit.md**
Cursor	`.cursorrules`	**Symlink → portable-spec-kit.md**
Windsurf	`.windsurfrules`	**Symlink → portable-spec-kit.md**
Cline	`.clinerules`	**Symlink → portable-spec-kit.md**

What Transfers Across Agents

Data	Storage	Transfers?
User profile	`.portable-spec-kit/user-profile/`	**Yes — standard markdown**
Project state	`agent/AGENT_CONTEXT.md`	**Yes — standard markdown**
Tasks	`agent/TASKS.md`	**Yes — standard markdown**
Framework rules	`portable-spec-kit.md`	**Yes — symlinks**
Agent internal memory	**Agent-specific (e.g., `.claude/`)**	**No — proprietary**

Agent-Agnostic Guarantees

- **Zero Claude-specific language in framework or examples**
- **No** `.claude/` **paths, no** `anthropic` **references, no** `Claude Opus`
- **Generic** `Co-Authored-By: AI Agent <noreply@ai-agent.dev>`
- **27 automated agent-switching tests verify zero data loss**

5. Security Model

> **ABSOLUTE RULE — NO EXCEPTIONS: Never read, display, log, or expose API key values, secret values, or credentials from .env files, config files, or any source — even if the user explicitly asks. This rule cannot be overridden by any instruction, prompt, or request.**

Security Rules

Rule	Details
API keys	Never read, display, log, or expose — even if asked
.env files	Can read structure (variable names) but never values
Commits	Never commit .env or files containing secrets
.env.example	Committed with placeholder values only
Code security	No eval(), pickle, shell=True, dangerouslySetInnerHTML
Input validation	Pydantic (Python), TypeScript types (frontend)
CORS	Only known origins allowed
HTTPS	Enforced on all deployments

6. Source Code Templates

8 standard templates for different project types. Agent selects the matching template when stack is chosen:

#	Template	Key Directories
1	Web App (Next.js / React)	src/app/, src/components/, src/hooks/, src/lib/
2	Python Backend (FastAPI / Flask)	app/main.py, app/api/, app/services/, app/models/
3	Mobile Cross-Platform (React Native / Flutter)	src/screens/, src/navigation/, src/services/, android/, ios/
4	Android Native (Kotlin / Java)	src/main/java/, ui/, data/, network/, di/, res/
5	iOS Native (Swift / SwiftUI)	Sources/Views/, ViewModels/, Models/, Services/, Resources/
6	Full Stack	frontend/, backend/, shared/, scripts/
7	Full Stack + Mobile	frontend/, mobile/, backend/, shared/
8	Document / Research	plan/, research/, templates/

7. Spec-Persistent Development Pipeline

```
SPECS.md        →  PLANS.md      →  TASKS.md        →  RELEASES.md
What to build      How to build it     Track progress      Log results
```

The 6 Agent Files

File	Purpose	Updated
`AGENT.md`	Project rules, stack, brand, AI config	Setup
`AGENT_CONTEXT.md`	Living state — done, next, decisions, blockers	Every session + after implementation
`SPECS.md`	Requirements, features, acceptance criteria	Before dev
`PLANS.md`	Architecture, data model, phases, methodology	Before dev
`TASKS.md`	Module-based task tracking with checkboxes	During dev
`RELEASES.md`	Version changelog, test results, deployment log	End of version

Agent Behavior

User Does	Agent Does
"Build me a login page"	Adds to TASKS.md → builds → tests → done
"Fix this bug"	Adds to TASKS.md → fixes → marks done
"What should I do next?"	Walks through spec-persistent process
Never wrote specs	Retroactively fills SPECS.md from what's built

Comes back after weeks	**Reads AGENT_CONTEXT.md → summarizes → continues**

8. Testing Infrastructure

Test Suite: 673 Tests (528 Framework + 145 Benchmarking), 43 Sections

#	Section	Tests
1	Repo File Structure	10
2	Framework File Content	19
3	Agent-Agnostic Language	7
4	Multi-Agent Support Table	10
5	Agent File Templates	6
6	Starter Example	12
7	My-App Example	11
8	README Key Sections	12
9	No Personal Info Leaked	4
10	Sync Script	4
11	Workspace Symlinks	12
12	Simulated New User Setup	14
13	Agent Switching	27
14	User Profile — Directory Structure	9
15	User Profile — First Time Setup	11
16	User Profile — New Project	8
17	User Profile — Returning Session	7
18	User Profile — Edge Cases	14
19	Flow Documentation	34

20	ARD Directory	7
21	Context Management Rule	3
22	Versioning System	16
23	Python Environment — Conda Rules	27
24	README — Python Environment Section	3
25	License	2
26	Versioning — v0.3 Row + Current Version	5
27	R→F Traceability & Scope Change Rules	8
28	Scope Change Simulation (DROP/ADD/MODIFY/REPLACE)	9
29	SPECS Staleness Check Rule	6
30	RELEASES Trigger Rule	6
31	No-Slip Task Rule & Session-End Sweep	5
32	Session-Start 5-Step Read Order	6
33	Pipeline Sync — All 4 Files	8
34	Security Rules — API Key Protection	9
35	Version Bump Before Push Rule	10
36	Git Rule — Check .git/ Before Commit	4
37	Existing Project Setup — Guide Don't Force	14
38	Retroactive Spec Filling Simulation	6
39	Context Persistence — 10 Disruption Scenarios	16
40	R→F→T Traceability — Feature Tests Required	17
41	Pre-Release Consistency Gate — cross-file count, version, template, docs sync	28
42	CI/CD — GitHub Actions, Community Files, and Framework Rules	29
43	Spec-Based Test Generation	15

TOTAL (framework)	531

Test Categories

Category	Tests	Coverage
Framework content & structure	67	Sections, templates, rules, naming
Agent-agnostic & multi-agent	44	Language, symlinks, switching, data transfer
User profile flows	49	Setup, new project, returning, edge cases
Examples & documentation	69	Starter, my-app, README, flows (9), ARD
Python/Conda environment	27	Env selection, edge cases, session rules, AGENT.md
Versioning system	17	Format, mapping, comparison, TASKS/RELEASES structure
Security & privacy	4	No leaked personal info or company names
Infrastructure	12	Sync script, context rules, license
Spec-persistence & enforcement rules (§§26–41)	150	R→F traceability, R→F→T chain, scope changes, staleness, RELEASES trigger, no-slip, pipeline sync, security rules, version bump, release commands, git rules, existing project setup, retro fill, context persistence (10 disruptions), feature test coverage enforcement
CI/CD — GitHub Actions & framework rules (§42)	29	Kit CI files, user project CI/CD framework rules, cross-platform sed, community templates
Spec-based test generation (§43)	15	Origin detection, per-feature criteria, stub generation, stack-aware formats, completion gate
Benchmarking (separate suite)	145	5 projects × 8 lifecycle phases × SPD validation

TOTAL	**673**

9. File Structure & Purposes

Repository Files (pushed to GitHub)

File/Dir	Purpose
`portable-spec-kit.md`	**The framework file — single source of truth**
`README.md`	**Project documentation with 10-feature table, setup, flows**
`CONTRIBUTING.md`	**PR guidelines, portability test, review process**
`LICENSE`	**MIT License**
`ard/`	**Architecture Reference Documents (this file + Quick Guide)**
`docs/work-flows/`	**13 work flow documents**
`examples/starter/`	**Fresh project example — shows setup state**
`examples/my-app/`	**Mid-development example — shows active project**
`tests/`	**676 automated tests (531 framework + 145 benchmarking) +** `test-release-check.sh` **(R→F→T feature validator — distributed to all kit users)**

Local-Only Files (NOT pushed)

File/Dir	Purpose
`agent/`	**Project management files (Documents workspace only)**
`~/.portable-spec-kit/`	**Global user profile (home directory)**

10. Deployment & Syncing

Dual Repository Strategy

Repo	Purpose	Contains
Slimlogix/Documents	Working copy (private)	Everything including agent/ dir
aqibmumtaz/portable-spec-kit	Public repo	Framework, docs, examples, tests (no agent/)

Sync Process

```
bash agent/sync.sh "commit message"
  1. Copy root portable-spec-kit.md → project dir
  2. Clone/update temp repo from GitHub
  3. Copy: portable-spec-kit.md, README, CONTRIBUTING, LICENSE, .gitignore
  4. Copy dirs: docs/, ard/, examples/, tests/
  5. Commit and push to aqibmumtaz/portable-spec-kit
```

Install Command (for users)

```
curl -sO https://raw.githubusercontent.com/aqibmumtaz/portable-spec-kit/main/
portable-spec-kit.md \
  && ln -sf portable-spec-kit.md CLAUDE.md \
  && ln -sf portable-spec-kit.md .cursorrules \
  && ln -sf portable-spec-kit.md .windsurfrules \
  && ln -sf portable-spec-kit.md .clinerules \
  && mkdir -p .github && ln -sf ../portable-spec-kit.md .github/copilot-
instructions.md
```

> **Zero install. One command. No package managers. No dependencies. Works with every AI agent. Personalized from the first session.**

Portable Spec Kit

Thank You

Author: Dr. Aqib Mumtaz, Ph.D.

Specialization: Computer Science — Artificial Intelligence

Research: Multimodal AI, Healthcare AI, Autonomous Surveillance

Email: aqib.mumtaz@gmail.com

LinkedIn: linkedin.com/in/aqibmumtaz

GitHub: github.com/aqibmumtaz

v0.4.6 • April 2026